
定义

透明多级分流系统主要考虑流量分流设计，设计从客户端到服务端的整个网络链路优化，核心是各类缓存的设计。设计上主要考虑两个原则：

- 避免单点设计
- 奥卡姆剃刀原则，部件不是越多越好

缓存设计

客户端缓存

客户端缓存主要分为：

- 状态缓存
- 强制缓存
- 协商缓存

状态缓存

状态缓存是指不经过服务器，客户端直接根据缓存信息对目标网站的状态判断。^{[1]*}
(也就是本地判断缓存状态)

强制缓存

HTTP的强制缓存对一致性问题的处理策略就如它的名字一样，十分直接：假设在某个时点到来以前，譬如收到响应后的10分钟内，资源的内容和状态一定不会被改变，因此客户端可以无须经过任何请求，在该时点前一直持有和使用该资源的本地缓存副本。^[2]

强制缓存主要有以下方案：

Expires

Expires是HTTP/1.0协议中开始提供的Header，后面跟随一个截止时间参数。当服务器返回某个资源时带有该Header，意味着服务器承诺资源在截止时间之前不会发生变

动，浏览器可直接缓存该数据，不再重新发请求^[2-1]。

```
HTTP/1.1 200 OK
Expires: Wed, 8 Apr 2020 07:28:00 GMT
```

但有以下几个缺陷^[2-2]：

- **受限于客户端的本地时间。**譬如，在收到响应后，客户端修改了本地时间，将时间前后调整几分钟，就可能会造成缓存提前失效或超期持有。
- **无法处理涉及用户身份的私有资源。**譬如，某些资源被登录用户缓存在自己的浏览器上是合理的，但如果被代理服务器或者内容分发网络缓存起来，则可能被其他未认证的用户所获取。
- **无法描述“不缓存”的语义。**譬如，浏览器为了提高性能，往往会自动在当次会话中缓存某些MIME类型的资源，在HTTP/1.0的服务器中就缺乏强制手段不允许浏览器缓存某个资源。以前为了实现这类功能，通常不得不使用脚本，或者手工在资源后面增加时间戳（譬如“xx.js?t=1586359920”、“xx.jpg?t=1586359350”）来保证每次资源都会重新获取。

Cache-Control

Cache-Control是HTTP/1.1协议中定义的强制缓存Header，它的语义比Expires丰富了很多，如果Cache-Control和Expires同时存在，并且语义存在冲突（譬如Expires与max-age/s-maxage冲突）的话，规定必须以Cache-Control为准^[2-3]。

Cache-Control包含以下几个关键参数：

- **max-age和s-maxage：**max-age后面跟随一个以秒为单位的数字，表明相对于请求时间（在Date Header中会注明请求时间）多少秒以内缓存是有效的，即多少秒以内不需要重新从服务器中获取资源。相对时间避免了Expires中采用的绝对时间可能受客户端时钟影响的问题。s-maxage中的“s”是“share”的缩写，意味“共享缓存”的有效时间，即允许被CDN、代理等持有的缓存有效时间，用于提示CDN这类服务器应在何时让缓存失效。
- **public和private：**指明是否涉及用户身份的私有资源，如果是public，则可以被代理、CDN等缓存；如果是private，则只能由用户的客户端进行私有缓存。

- **no-cache和no-store**：no-cache指明该资源不应该被缓存，哪怕是同一个会话中对同一个URL地址的请求，也必须从服务端获取，**令强制缓存完全失效，但此时下一节中的协商缓存机制依然是生效的**；no-store不强制会话中相同URL资源的重复获取，但禁止浏览器、CDN等以任何形式保存该资源。
- **no-transform**：禁止以任何形式修改资源。譬如，某些CDN、透明代理支持自动GZip压缩图片或文本，以提升网络性能，而no-transform禁止了这样的行为，它不允许Content-Encoding、Content-Range、Content-Type进行任何形式的修改。
- **min-fresh和only-if-cached**：这两个参数是仅用于客户端的请求Header。min-fresh后面跟随一个以秒为单位的数字，用于建议服务器能返回一个不少于该时间的缓存资源（即包含max-age且不少于min-fresh的数字）。only-if-cached表示客户端要求不给它发送资源的具体内容，此时客户端仅能使用事先缓存的资源来进行响应，若缓存不能命中，就直接返回503/Service Unavailable错误。
- **must-revalidate和proxy-revalidate**：must-revalidate表示在资源过期后，一定要从服务器中进行获取，即超过了max-age的时间后，就等同于no-cache的行为，proxy-revalidate用于提示代理、CDN等设备资源过期后的缓存行为，除对象不同外，语义与must-revalidate完全一致。

I 协商缓存

强制缓存是基于时效性的，但无论是人还是服务器，其实多数情况下并没有什么把握去承诺某项资源多久不会发生变化。另外一种基于变化检测的缓存机制，在一致性上会有比强制缓存更好的表现，但需要一次变化检测的交互开销，性能上就会略差一些，这种基于检测的缓存机制，通常被称为“协商缓存”。^[3]

在HTTP中的协商缓存与强制缓存并没有互斥性，这两套机制是并行工作的。^[3-1]

协商缓存主要有以下几个关键机制实现（根据时间变动和资源唯一值变动）：

I Last-Modified和If-Modified-Since

Last-Modified是服务端的响应Header，用于告诉客户端这个资源的最后修改时间。对于带有这个Header的资源，当客户端需要再次请求时，会通过If-Modified-Since把之前收到的资源最后修改时间发送回服务端。^[3-2]

如果此时服务端发现资源在该时间后没有被修改过，就返回一个304/Not Modified的响应，无须附带消息体，即可达到节省流量的目的。^[3-3]

```
HTTP/1.1 304 Not Modified
Cache-Control: public, max-age=600
Last-Modified: Wed, 8 Apr 2020 15:31:30 GMT
```

如果此时服务端发现资源在该时间之后有变动，就会返回200/OK的完整响应，在消息体中包含最新的资源。[3-4]

```
HTTP/1.1 200 OK
Cache-Control: public, max-age=600
Last-Modified: Wed, 8 Apr 2020 15:31:30 GMT
Content
```

ETag和If-None-Match

ETag是服务端的响应Header，用于告诉客户端这个资源的唯一标识。HTTP服务端可以根据自己的意愿来选择如何生成这个标识，譬如Apache服务端的ETag值默认是对文件的索引节点(Inode)、大小和最后修改时间进行哈希计算后得到的。对于带有这个Header的资源，当客户端需要再次请求时，会通过If-None-Match把之前收到的资源唯一标识发送回服务端。[3-5]

如果此时服务端计算后发现资源的唯一标识与上传回来的标识一致，说明资源没有被修改过，就返回一个304/Not Modified的响应，无须附带消息体，即可达到节省流量的目的。[3-6]

```
HTTP/1.1 304 Not Modified
Cache-Control: public, max-age=600
ETag: "28c3f612-ceb0-4ddc-ae35-791ca840c5fa"
```

如果此时服务端发现资源的唯一标识有变动，就会返回200/OK的完整响应，在消息体中包含最新的资源。[3-7]

```
HTTP/1.1 200 OK
Cache-Control: public, max-age=600
ETag: "28c3f612-ceb0-4ddc-ae35-791ca840c5fa"
Content
```

ETag是性能最差的缓存机制，每次都要进行hash计算进行对比。

ETag和Last-Modified是允许一起使用的，服务端会优先验证ETag，在ETag一致的情况下，再去对比Last-Modified，这是为了防止有一些HTTP服务端未将文件修改日期纳入哈希范围内。[\[3-8\]](#)

根据约定，协商缓存不仅在浏览器的地址输入、页面链接跳转、新开窗口、前进、后退中生效，而且在用户主动刷新页面(F5)时同样是生效的，只有用户强制刷新(Ctrl+F5)或者明确禁用缓存（譬如在DevTools中设定）时才会失效，此时客户端向服务端发出的请求会自动带有“Cache-Control:no-cache”。[\[3-9\]](#)

域名解析

按个人理解就是通过DNS服务解析成IP，DNS域名解析有几个特点：

- 从本地出发，逐级查询
- 某个域名更换了IP，没有任何机制通知其他DNS服务器更新了，只能等它TTL过期

避免DNS劫持的一种新技术：

HTTPDNS（也称为DNS over HTTPS，DoH）。它将原本的DNS解析服务开放为一个基于HTTPS协议的查询服务，替代基于UDP传输协议的DNS域名解析，通过程序代替操作系统直接从权威DNS或者可靠的本地DNS获取解析数据，从而绕过传统本地DNS。这样做的好处是完全免去了“中间商赚差价”的环节，不再惧怕底层的域名劫持，有效避免本地DNS不可靠导致的域名生效缓慢、来源IP不准确、产生的智能线路切换错误等问题[\[4\]](#)。

传输链路

传输链路优化主要考虑以下几个方面：

连接数优化

在HTTP 1.x的时代，主要使用持久连接Keep-alive技术来保持TCP链路复用，从而避免反复创建TCP连接。

但Keep-alive有明显缺点，那就是队首阻塞。请设想以下场景：浏览器有10个资源需要从服务器中获取，此时它将10个资源放入队列，入列顺序只能按照浏览器遇见这些资源的先后顺序来决定。但如果这10个资源中的第1个就让服务器陷入长时间运算状态会怎样呢？当它的请求被发送到服务端之后，服务端开始计算，而运算结果出来之前TCP连接中并没有任何数据返回，此时后面9个资源都必须阻塞等待。因为服务端虽然可以并行处理另外9个请求（譬如第1个是复杂运算请求，消耗CPU资源，第2个是数据库访问，消耗数据库资源，第3个是访问某张图片，消耗磁盘I/O资源，这就很适合并行），但问题是处理结果无法及时返回客户端，服务端不能因为哪个请求先完成就返回哪个，更不可能将所有要返回的资源混杂到一起交叉传输，原因是只使用一个TCP连接来传输多个资源的话，如果顺序乱了，客户端就很难区分哪个数据包归属哪个资源了。^[5]（简单理解就是只可串行化，无法并发。如果并发，次序就乱了）

队首阻塞问题一直持续到第二代的HTTP协议，即HTTP/2发布后才算是被比较完美地解决。在HTTP/1.x中，HTTP请求就是传输过程中最小粒度的信息单位了，所以如果将多个请求切碎，再混杂在一块传输，客户端势必难以分辨、重组出有效信息。而在HTTP/2中，帧(Frame)才是最小粒度的信息单位，它可以用来描述各种数据，譬如请求的Headers、Body，或者用来做控制标识，譬如打开流、关闭流。这里说的流(Stream)是一个逻辑上的数据通道概念，每个帧都附带一个流ID以标识这个帧属于哪个流。这样，在同一个TCP连接中传输的多个数据帧就可以根据流ID轻易区分开来，在客户端毫不费力地将不同流中的数据重组出不同HTTP请求和响应报文来。这项设计是HTTP/2的最重要的技术特征一，被称为HTTP/2多路复用(HTTP/2 Multiplexing)技术。^[5-1]

HTTP/2就可以对每个域名只维持一个TCP连接(One Connection Per Origin)并以任意顺序传输任意数量的资源了，这样既减轻了服务器的连接压力，也不需要开发者去考虑域名分片这种事情来突破浏览器对每个域名最多6个的连接数限制。没有TCP连接数的压力，就无须刻意压缩HTTP请求，所有通过合并、内联文件（无论是图片、样式、脚本）以减少请求数的需求都不再成立，甚至会被当作徒增副作用的反模式。^[5-2]

相对HTTP1，HTTP有几个要注意的特点^[5-3]：

- HTTP/2是单域名单连接的机制，合并资源和域名分片反而不利于提升Header压缩效果
- 与HTTP/1.x相比，HTTP/2本身变得更适合传输小资源。

I 传输压缩

在HTTP 1.0中，持久传输Keep-alive与即时压缩存在冲突，使用了即时压缩的话无法知道content-length，也就无法判定某个资源传输结束。在HTTP 1.1中引入了分块传输编码（Chunked Transfer Encoding）机制，解决了持久传输与即时压缩的冲突^[6]。

分块编码的原理相当简单：在响应Header中加入“Transfer-Encoding:chunked”之后，就代表这个响应报文将采用分块编码。此时，报文中的Body需要改为用一系列“分块”来传输。每个分块包含十六进制的长度值和对应长度的数据内容，长度值独占一行，数据从下一行开始，最后以一个长度值为0的分块来表示资源结束。举个具体例子^[6-1]：

```
HTTP/1.1 200 OK
Date: Sat, 11 Apr 2020 04:44:00 GMT
Transfer-Encoding: chunked
Connection: keep-alive
25
This is the data in the first chunk
1C
and this is the second one
3
con
8
sequence
0
```

根据分块长度可知，前两个分块包含显式的回车换行符（CRLF，即\r\n字符）。

"This is the data in the first chunk\r\n"	(37 字符 => 十六进制: 0x25)
"and this is the second one\r\n"	(28 字符 => 十六进制: 0x1C)
"con"	(3 字符 => 十六进制: 0x03)
"sequence"	(8 字符 => 十六进制: 0x08)

所以解码后的内容为：

```
This is the data in the first chunk
and this is the second one
consequence
```

传输压缩在HTTP 2中仍然具有不少价值。

I 快速UDP网络连接

QUIC底层使用UDP替代TCP，性能会更高。而HTTP 3则将QUIC作为传输层，在其基础上实现HTTP协议。

简单理解就是HTTP 2是基于TCP，而HTTP 3是基于QUIC。

HTTP3 是未来趋势。

I 内容分发网络

内容分发网络就是CDN，这个之前设计比较少，简单过一下就好。

CDN主要涉及以下几个内容：

I 路由解析

路由解析关系到CDN的响应时间，客户端通过路由解析找到响应最快的CDN，从而提升整体响应速度。

具体命令主要是 `dig xxx.com` ^[7]。

I 内容分发

CDN主要涉及两种分发方式（将资源部署到CDN）[\[8\]](#)：

- **主动分发(Push)**：分发由源站主动发起，将内容从源站或者其他资源库推送到用户边缘的各个CDN缓存节点上。这个推送的操作没有什么业界标准可循，可以选择任何传输方式（HTTP、FTP、P2P，等等）、任何推送策略（满足特定条件、定时、人工，等等）、任何推送时间，只要与后面说的更新策略相匹配即可。由于主动分发通常需要源站、CDN服务双方提供程序API接口层面的配合，所以它对源站并不是透明的，只对用户一侧单向透明。主动分发一般用于网站要预载大量资源的场景。譬如在双十一之前的一段时间内，淘宝、京东等各个网络商城会把未来活动中所要用的资源推送到CDN缓存节点中，特别常用的资源甚至会直接缓存到你的手机App的存储空间或者浏览器的localStorage上。
- **被动回源(Pull)**：被动回源由用户访问所触发，是全自动、双向透明的资源缓存过程。当某个资源首次被用户请求的时候，若CDN缓存节点发现自己没有该资源，就会实时从源站中获取，这时资源的响应时间可粗略认为是资源从源站到CDN缓存节点的时间，加上资源从CDN发送到用户的时间之和。因此，被动回源的首次访问通常比较慢（但由于CDN的网络条件一般远高于普通用户，并不一定比用户直接访问源站更慢），不适合应用于数据量较大的资源。被动回源的优点是可以做到完全的双向透明，不需要源站在程序上做任何配合，使用起来非常方便。这种分发方式是小型站点使用CDN服务的主流选择，如果不是自建CDN，而是购买阿里云、腾讯云的CDN服务的站点，多数采用的就是这种方式。

CDN如何管理（更新）资源？[\[8-1\]](#)

最常见的做法是超时被动失效与手工主动失效相结合。超时被动失效是指给予缓存资源一定的生存期，超过了生存期就在下次请求时重新被动回源一次。而手工主动失效是指CDN服务商一般会提供处理失效缓存的接口，在网站更新时，由持续集成的流水线自动调用该接口来实现缓存更新，譬如“icyfenix.cn”就是依靠Travis-CI的持续集成服务来触发CDN失效和重新预热的。[\[8-2\]](#)

CDN应用

CDN现在有以下广发应用[\[9\]](#)：

- 加速静态资源分发：这是CDN的本职工作。
- 安全防御：CDN在广义上可以视作网站的堡垒机，源站只对CDN提供服务，由CDN来对外界其他用户提供服务，这样恶意攻击者就不容易直接威胁源站。CDN

对某些攻击手段的防御，如对DDoS攻击的防御尤其有效。但需注意，将安全都寄托在CDN上本身是不安全的，一旦源站真实IP被泄漏，就会面临很高的风险。

- 协议升级：不少CDN提供商都同时对接（代售CA的）SSL证书服务，可以实现源站是基于HTTP协议的，而对外开放的网站是基于HTTPS的。同理，可以实现源站到CDN是HTTP/1.x协议，CDN提供的外部服务是HTTP/2或HTTP/3协议；实现源站是基于IPv4网络的，CDN提供的外部服务支持IPv6网络，等等。
- 状态缓存：4.1节介绍客户端缓存时简要提到了状态缓存，而CDN不仅可以缓存源站的资源，还可以缓存源站的状态，譬如可以通过CDN缓存源站的301/302状态让客户端直接跳转，也可以通过CDN开启HSTS、通过CDN进行OCSP装订加速SSL证书访问，等等。有一些情况下甚至可以配置CDN对任意状态码（譬如404）进行一定时间的缓存，以减轻源站压力，但这个操作应当慎重，且在网站状态发生改变时要及时刷新缓存。
- 修改资源：CDN可以在返回资源给用户的时候修改资源的任何内容，以实现不同的目的。譬如，可以对源站未压缩的资源自动压缩并修改Content-Encoding，以节省用户的网络带宽消耗，可以对源站未启用客户端缓存的内容加上缓存Header，自动启用客户端缓存，可以修改CORS的相关Header，为源站不支持跨域的资源提供跨域能力，等等。
- 访问控制：CDN可以实现IP黑/白名单功能，如根据不同的来访IP提供不同的响应结果，根据IP的访问流量来实现QoS控制，根据HTTP的Referer来实现防盗链，等等。
- 注入功能：CDN可以在不修改源站代码的前提下，为源站注入各种功能。图4-7所示是国际CDN巨头CloudFlare提供的Google Analytics、PACE、Hardenize等第三方应用，这些原本需要在源站中注入代码的应用，在有CDN参与的情况下均能做到无须修改源站任何代码即可使用。

负载均衡

无论在网关内部建立了多少级的负载均衡，从形式上来说都可以分为两种：四层负载均衡和七层负载均衡。有以下两个原则^[10]：

- 四层负载均衡的优势是性能高，七层负载均衡的优势是功能强。
- 做多级混合负载均衡，通常应是低层负载均衡在前，高层负载均衡在后。

所谓“四层”和“七层”一般是指OSI模型中的四层和七层，一般指的是工作在哪个层。但这里所说的“四层负载均衡”其实是多种均衡器工作模式的统称，“四层”是说这些工作模式的共同特点是维持同一个TCP连接，而不是说它只工作在第四层。

表4-1 OSI七层模型

#	层	数据单元	功能
七	应用层 (Application Layer)	数据 (Data)	为应用软件提供服务的接口，用于与其他应用软件之间的通信。典型协议有 HTTP、HTTPS、FTP、Telnet、SSH、SMTP、POP3 等
六	表达层 (Presentation Layer)	数据 (Data)	把数据转换为能与接收者的系统格式兼容并适合传输的格式
五	会话层 (Session Layer)	数据 (Data)	负责在数据传输中设置和维护计算机网络中两台计算机之间的通信连接
四	传输层 (Transport Layer)	数据段 (Segment)	把传输表头加至数据后以形成数据包。传输表头包含所使用的协议等发送信息。典型协议有 TCP、UDP、RDP、SCTP、FCP 等

(续)

#	层	数据单元	功能
三	网络层 (Network Layer)	数据包 (Packet)	提供数据的传输路径选择和转发功能，将网络表头附加至数据段后以形成报文（即数据包）。典型协议有 IPv4/IPv6、IGMP、ICMP、EGP、RIP 等
二	数据链路层 (Data Link Layer)	数据帧 (Frame)	负责点对点的网络寻址、错误侦测和纠错。当表头和表尾被附加至数据包后，就形成数据帧（Frame）。典型协议有 Wi-Fi (802.11)、Ethernet (802.3)、PPP 等
一	物理层 (Physical Layer)	比特流 (Bit)	在局域网上传送数据帧，负责管理电脑通信设备和网络媒体之间的互通。包括针脚、电压、线缆规范、集线器、中继器、网卡、主机接口卡等

对于负载均衡的设计，有以下几个手段：

I 数据链路层负载均衡

原理：只修改源和目的的mac地址。

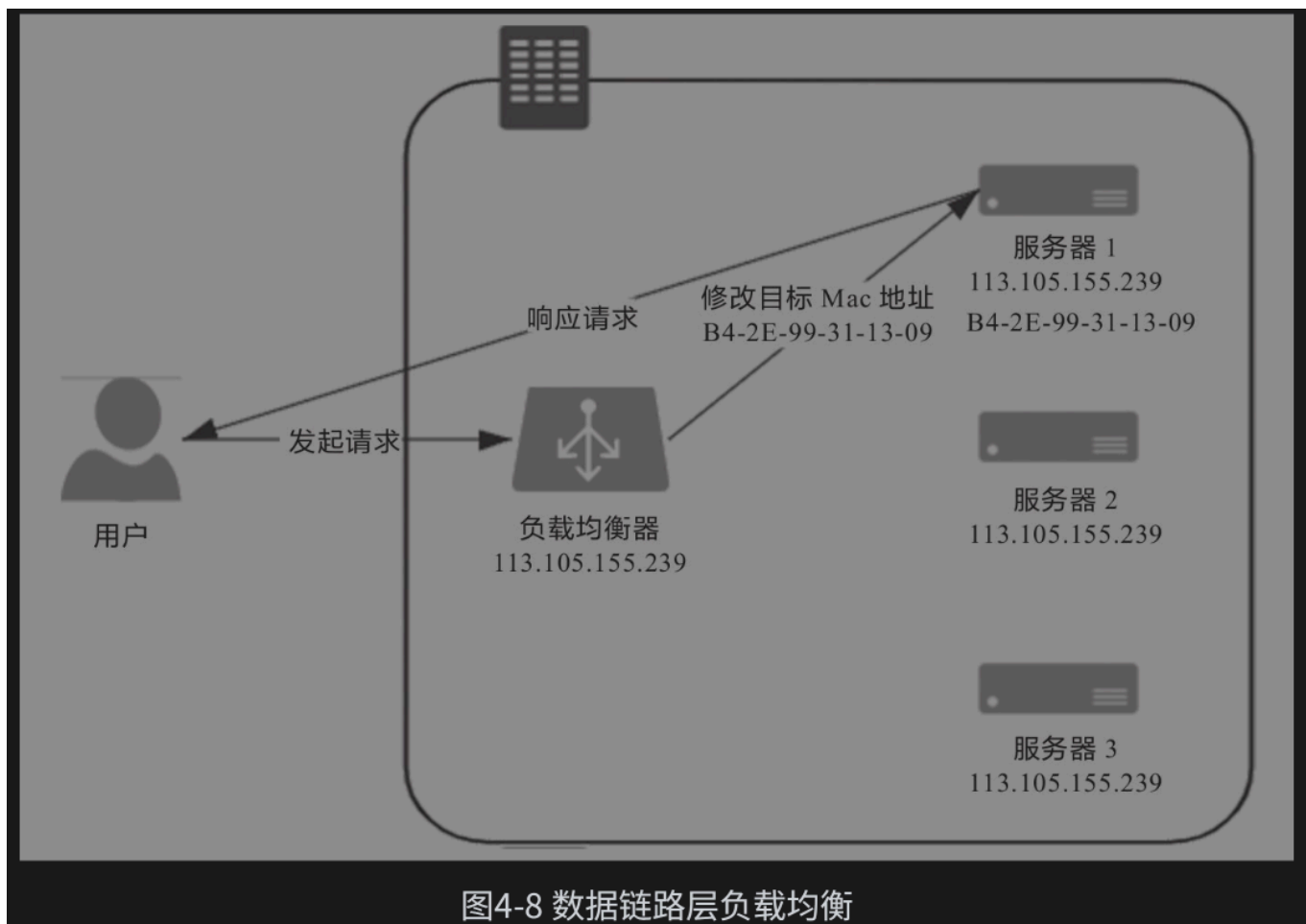


图4-8 数据链路层负载均衡

在上述只有请求经过负载均衡器，而服务的响应无须从负载均衡器原路返回的工作模式中，整个请求、转发、响应的链路形成了一个“三角关系”，所以这种负载均衡模式也常被形象地称为“三角传输模式”(Direct Server Return, DSR)，也称为“单臂模式”(Single Legged Mode)或者“直接路由”(Direct Routing)^[11]。

这种负载均衡模式有以下特点：

- 优势是效率高性能好。（不需经过网络层和应用层干预）
- 劣势是由于修改的是二层，所以负载均衡器必须与真实服务器在同一个子网中，无法跨vlan，也不能感知网络层和应用层的信息。

网络层负载均衡

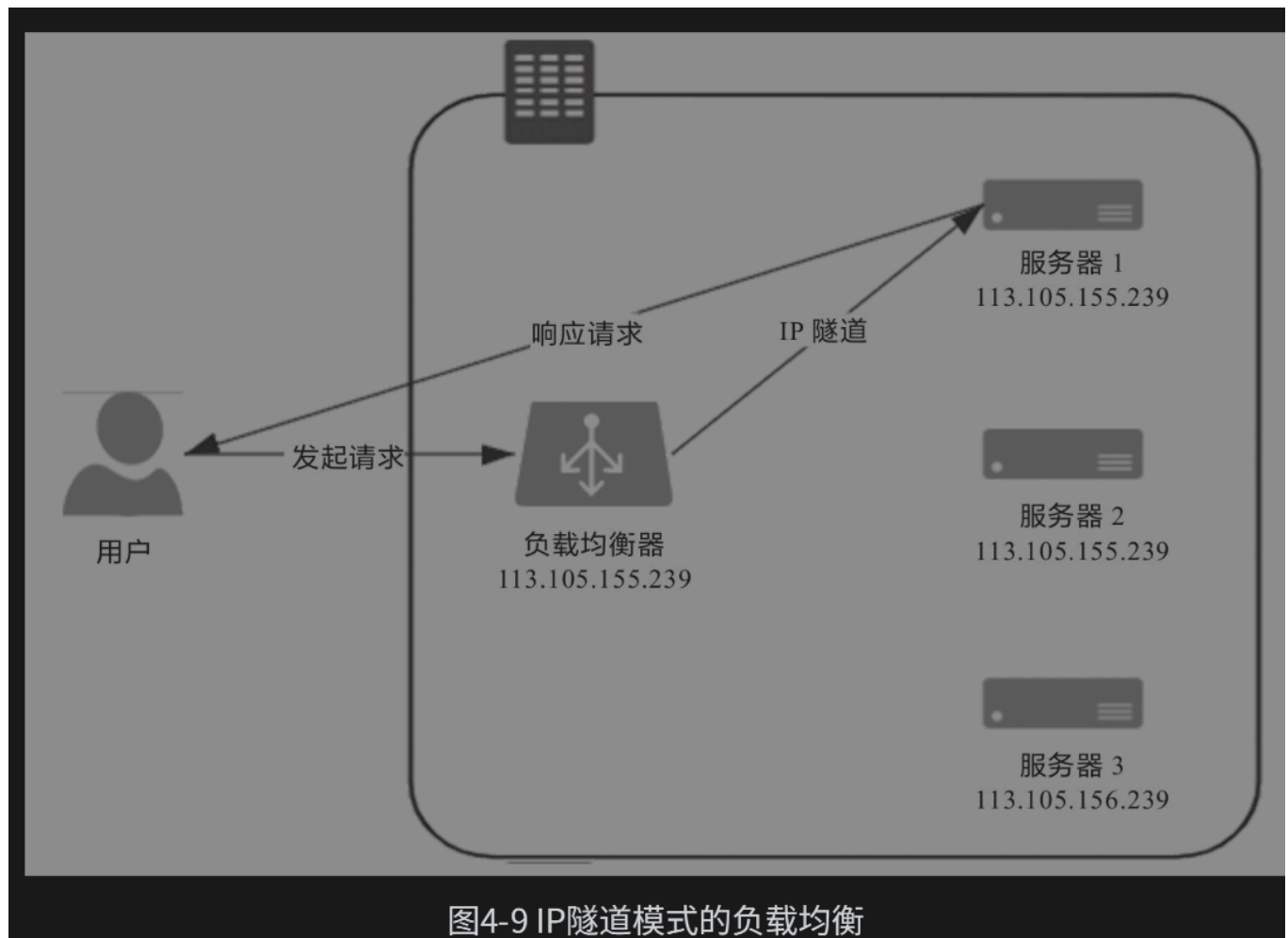
原理：只修改源和目的IP地址。

网络层负载均衡有两种实现方式：IP隧道和NAT模式。

IP隧道

原理：保持原数据包不变，新创建一个数据包，把原数据包的Header和Payload整体作为新数据包的Payload，并在这个新数据包的Header中写入真实服务器的IP作为目标地址，然后把它发送出去。经过三层交换机的转发，真实服务器收到数据包后，必须在接收入口处设计一个针对性的拆包机制，把由负载均衡器自动添加的那层Header扔掉，还原出原数据包来进行使用。这样，真实服务器就同样拿到了一个原本不是发给它（目标IP不是它）的数据包，达到了流量转发的目的。当时还没有流行起“禁止套娃”的梗，所以设计者将这种“套娃式”的传输称为“IP隧道”(IP Tunnel)传输，也是相当的形象。[\[12\]](#)

尽管因为要封装新的数据包，IP隧道转发模式的效率比直接路由模式的效率低，但由于并没有修改原数据包中的任何信息，所以IP隧道转发模式仍然具备三角传输的特性，即负载均衡器转发来的请求，可以由真实服务器去直接应答，无须经过负载均衡器原路返回。而且由于IP隧道工作在网络层，所以可以跨越VLAN，因此摆脱了直接路由模式中网络侧的约束。[\[12-1\]](#)



这种方式有几个缺点：

- 第一个缺点是它要求真实服务器必须支持“IP隧道协议”(IP Encapsulation)，即它得学会自己拆包扔掉一层Header，这个其实并不是什么大问题，现在几乎所有的Linux系统都支持IP隧道协议。[\[12-2\]](#)
- 第二个缺点是这种模式仍必须通过专门的配置，必须保证所有的真实服务器与负载均衡器有相同的虚拟IP地址，因为回复该数据包时，需要使用这个虚拟IP作为响应数据包的源地址，这样客户端收到这个数据包时才能正确解析。[\[12-3\]](#)

I NAT模式

原理：改变目标数据包：直接修改数据包Header中的目标地址，修改后原本由用户发给负载均衡器的数据包也会被三层交换机转发到真实服务器的网卡上，而且因为没有经过IP隧道的额外包装，也就无须再拆包了。但问题是这种模式是通过修改目标IP地址才到达真实服务器的，如果真实服务器直接将应答包返回客户端，即这个应答数据包的源IP是真实服务器的IP，也即均衡器修改以后的IP地址，则客户端不可能认识该IP，自然就无法正常处理这个应答了。因此，只有让应答流量继续回到负载均衡器，由负载均衡器把应答包的源IP改回自己的IP，再发给客户端，才能保证客户端与真实服务器之间的正常通信。[\[12-4\]](#)

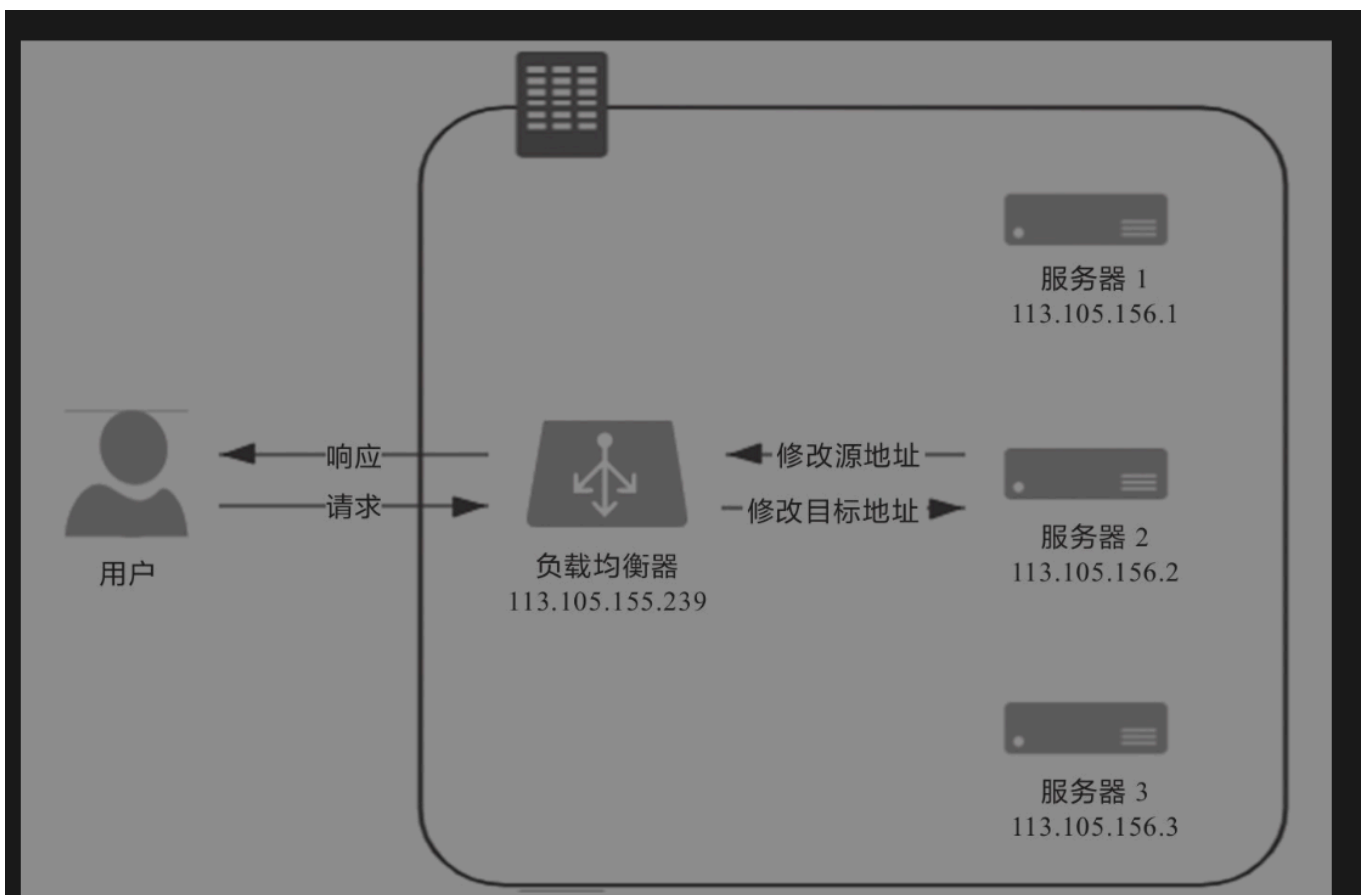


图4-10 NAT模式的负载均衡

在流量压力比较大的时候，NAT模式的负载均衡会带来较大的性能损失，比起直接路由和IP隧道模式，甚至会出现数量级上的下降。[\[12-5\]](#)

还有一种更加彻底的NAT模式：即负载均衡器在转发时，不仅会修改目标IP地址，还会修改源IP地址，这样源地址就改成负载均衡器自己的IP，称作Source NAT(SNAT)。这样做的好处是真实服务器无须配置网关就能够让应答流量经过正常的三层路由回到负载均衡器上，做到彻底的透明。但是缺点是由于做了SNAT，真实服务器处理请求时就无法拿到客户端的IP地址，从真实服务器的视角来看，所有的流量都来自于负载均衡器，这样有一些需要根据目标IP进行控制的业务逻辑就无法进行了。[\[12-6\]](#)

I 应用层负载均衡

前面介绍的四层负载均衡工作模式都属于“**转发**”，即直接将承载着TCP报文的底层数据格式（IP数据包或以太网帧）转发到真实服务器上，此时客户端与响应请求的真实服务器维持着同一条TCP通道。但工作在四层之后的负载均衡模式就无法再转发了，只能**代理**，此时真实服务器、负载均衡器、客户端三者之间由两条独立的TCP通道来维持通信。转发与代理的区别如图所示。[\[13\]](#)

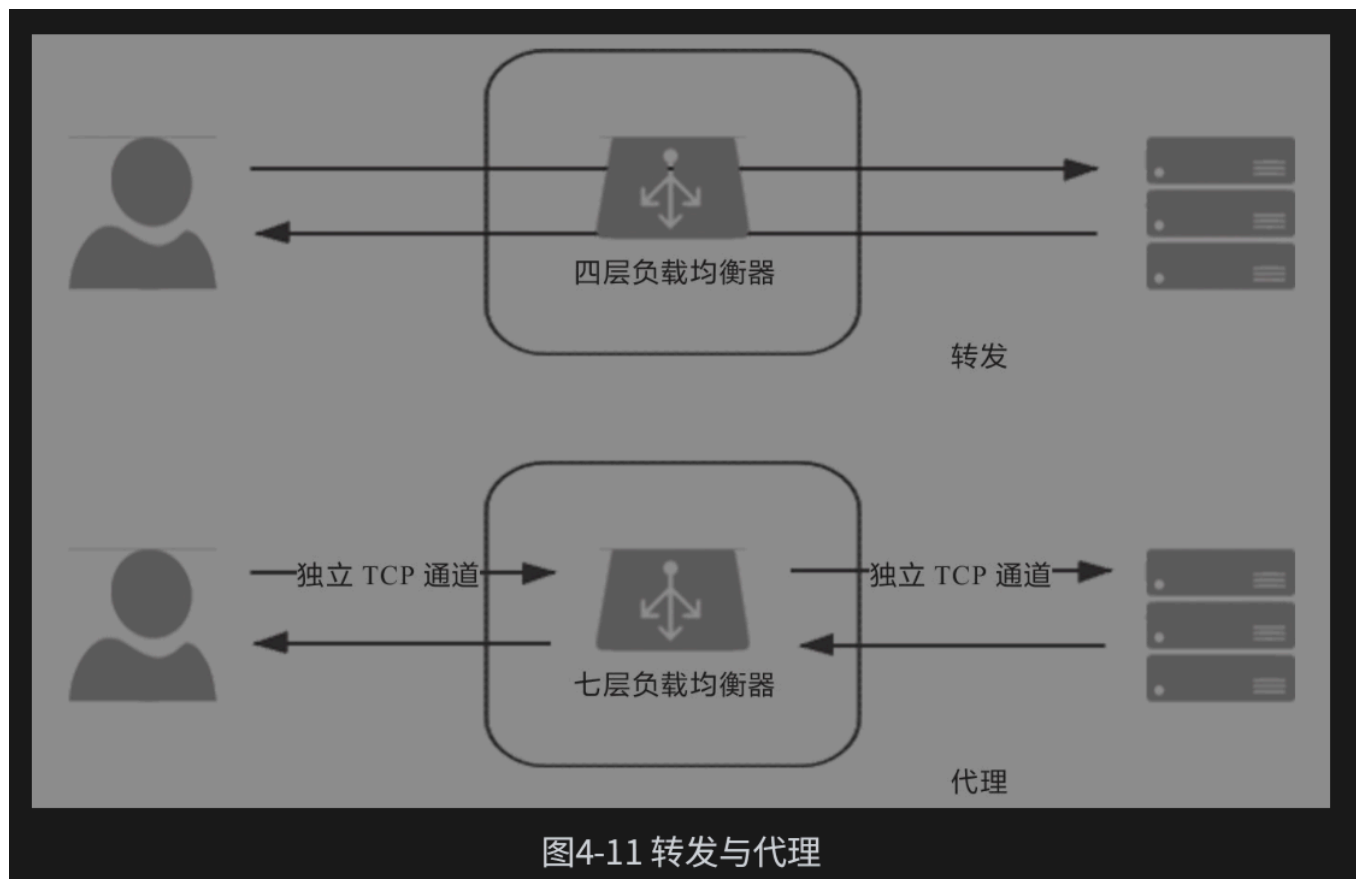


图4-11 转发与代理

“代理”这个词，根据“哪一方能感知到”的原则，可以分为“正向代理”“反向代理”和“透明代理”三类。正向代理就是我们通常简称的代理，指在客户端设置的，代表客户端与服

务器通信的代理服务，它是客户端可知，而对服务器透明的。反向代理是指在服务器侧设置的，代表真实服务器与客户端通信的代理服务，此时它对客户端来说是透明的。至于透明代理是指对双方都透明的，配置在网络中间设备上的代理服务，譬如，架设在路由器上的透明翻墙代理。^[13-1]

根据以上定义，很显然，七层负载均衡器属于反向代理的一种。

应用层代理可以玩出很多花样，例如CDN上面做WAF。

I 均衡策略与实现

负载均衡的两大职责是“选择谁来处理用户请求”和“将用户请求转发过去”。^[14]

上面的内容主要是后者，这里讨论前者，即均衡的策略^[14-1]。

- **轮询均衡(Round Robin)**: 每一次来自网络的请求轮流分配给内部的服务器，从1至N然后重新开始。此种均衡算法适合于服务器集群中的所有服务器都有相同的软硬件配置并且平均服务请求相对均衡的情况。
- **权重轮询均衡(Weighted Round Robin)**: 根据服务器的不同处理能力，给每个服务器分配不同的权值，使其能够接受相应权值数的服务请求。譬如：设置服务器A的权值为1，B的权值为3，C的权值为6，则服务器A、B、C将分别接收到10%、30%、60%的服务请求。此种均衡算法能确保高性能的服务器得到更多的使用率，避免低性能的服务器负载过重。
- **随机均衡(Random)**: 把来自客户端的请求随机分配给内部的多个服务器，在数据量足够大的场景下能达到相对均衡的分布。
- **权重随机均衡(Weighted Random)**: 此种均衡算法类似于权重轮询算法，不过在分配处理请求时是随机选择的过程。
- **一致性哈希均衡(Consistency Hash)**: 将请求中的某些数据（可以是MAC、IP地址，也可以是更上层协议中的某些参数信息）作为特征值来计算需要落在的节点，算法一般会保证同一个特征值每次都一定落在相同的服务器上。这里的一致性是指保证当服务集群某个真实服务器出现故障时，只影响该服务器的哈希值，而不会导致整个服务器集群的哈希键值重新分布。
- **响应速度均衡(Response Time)**: 负载均衡设备对内部各服务器发出一个探测请求（例如Ping），然后根据内部各服务器对探测请求的最快响应时间来决定哪一台服务器响应客户端的服务请求。此种均衡算法能较好地反映服务器的当前运行状态，但最快响应时间仅仅指的是负载均衡设备与服务器之间的最快响应时间，而不是客户端与服务器之间的最快响应时间。

- **最少连接数均衡(Least Connection)**: 客户端的每一次请求服务在服务器停留的时间可能会有较大差异,随着工作时间增加,如果采用简单的轮询或随机均衡算法,每一台服务器上的连接进程可能会产生极大的不平衡,并不能达到真正的负载均衡。最少连接数均衡算法会对内部需负载的每一台服务器有一个数据记录,记录当前该服务器正在处理的连接数量,当有新的服务连接请求时,将把当前请求分配给连接数最少的服务器,使均衡更加符合实际情况,使负载更加均衡。此种均衡策略适合长时处理的请求服务,

负载均衡器的实现分为“软件均衡器”和“硬件均衡器”两类。在软件均衡器方面,又分为直接建设在操作系统内核的均衡器和应用程序形式的均衡器两种。前者的代表是LVS(Linux Virtual Server),后者的代表有Nginx、HAProxy、KeepAlived等。前者性能会更好,因为无须在内核空间和应用空间中来回复制数据包;而后者的优势是选择广泛,使用方便,功能不受限于内核版本。在硬件均衡器方面,往往会直接采用应用专用集成电路(Application Specific Integrated Circuit, ASIC)来实现,有专用处理芯片的支持,避免操作系统层面的损耗,以达到最高的性能。这类均衡器的代表就是著名的F5和A10公司的负载均衡产品^[14-2]。

I 服务端缓存

缓存虽然是典型以空间换时间来提升性能的手段,但它的出发点是缓解CPU和I/O资源在峰值流量下的压力,“顺带”而非“专门”地提升响应性能。^[15]

I 缓存属性

设计缓存时,需要考虑以下几个方面的属性^[16]:

- **吞吐量**: 缓存的吞吐量使用OPS值(每秒操作数, Operation per Second, ops/s)来衡量,反映了对缓存进行并发读、写操作的效率,即缓存本身的工作效率高低。
- **命中率**: 缓存的命中率即成功从缓存中返回结果次数与总请求次数的比值,反映了引入缓存的价值高低,命中率越低,引入缓存的收益越小,价值越低。
- **扩展功能**: 即缓存除了基本读写功能外,还提供哪些额外的管理功能,譬如最大容量、失效时间、失效事件、命中率统计,等等。
- **分布式缓存**: 缓存可分为“进程内缓存”和“分布式缓存”两大类,前者只为节点本身提供服务,无网络访问操作,速度快但缓存的数据不能在各服务节点中共享,后者则相反。

I 吞吐量

缓存的吞吐量只在并发场景中才有统计的意义，因为若不考虑并发，即使是最原始的、以HashMap实现的缓存，访问效率也已经是常量时间复杂度（即 $O(1)$ ）。[16-1]

JDK 8改进之后的ConcurrentHashMap基本上就是你能找到的吞吐量最高的缓存容器。[16-2]

在这种并发读写的场景中，吞吐量受多方面因素的共同影响，譬如，怎样设计数据结构以尽可能避免数据竞争，存在竞争风险时怎样处理同步（主要有使用锁实现的悲观同步和使用CAS实现的乐观同步）、如何避免伪共享现象（False Sharing，这也算是典型缓存提升开发复杂度的例子）发生，等等。其中尽可能避免竞争是最关键的，无论如何实现同步都不会比无须同步更快。[16-3]

缓存中最主要的数据竞争源于读取数据，同时也会伴随着对数据状态的写入操作，而写入数据的同时，又会伴随着数据状态的读取操作。譬如，读取时要同时更新数据的最近访问时间和访问计数器的状态，以实现缓存的淘汰策略；又或者读取时要同时判断数据的超期时间等信息，以实现失效重加载等其他扩展功能。对以上伴随读写操作而来的状态维护操作，有两种可选择的处理思路。一种是以Guava Cache为代表的同步处理机制，即在访问数据时一并完成缓存淘汰、统计、失效等状态变更操作，通过分段加锁等优化手段来尽量减少竞争。另一种是以Caffeine为代表的异步日志提交机制，这种机制参考了经典的数据库设计理论，将数据的读、写过程看作日志（即对数据的操作指令）的提交过程。尽管日志也涉及写入操作，有并发的数据变更就必然面临锁竞争，但异步提交的日志已经将原本在Map内的锁转移到日志的追加写操作上，日志里腾挪优化的余地就比在Map中要大得多。[16-4]

I 命中率与淘汰策略

缓存空间有限，注定要淘汰部分落后数据以释放缓存，以下是常用的淘汰策略[16-5]：

- **FIFO(First In First Out)**：优先淘汰最早进入被缓存的数据。FIFO的实现十分简单，但一般来说它并不是优秀的淘汰策略，越是频繁被用到的数据，往往会越早存入缓存之中。如果采用这种淘汰策略，很可能会大幅降低缓存的命中率。
- **LRU(Least Recent Used)**：优先淘汰最久未被访问过的数据。LRU通常会采用HashMap加LinkedList的双重结构（如LinkedHashMap）来实现，以HashMap来提供访问接口，保证常量时间复杂度的读取性能，以LinkedList的链表元素顺序来表示数据的时间顺序，每次缓存命中时把返回对象调整到LinkedList开头，每次缓存淘汰时从链表末端开始清理数据。对大多数的缓存场景来说，LRU明显要比

FIFO策略合理，尤其适合用来处理短时间内频繁访问的热点对象。但是如果一些热点数据在系统中被频繁访问，只是最近一段时间因为某种原因未被访问过，那么这些热点数据此时就会有被LRU淘汰的风险，换句话说，LRU依然可能错误淘汰价值更高的数据。

- **LFU(Least Frequently Used)**: 优先淘汰最不经常使用的数据。LFU会给每个数据添加一个访问计数器，每访问一次就加1，需要淘汰时就清理计数器数值最小的那批数据。LFU可以解决上面LRU中热点数据间隔一段时间不访问就被淘汰的问题，但同时它又引入了两个新的问题。第一个问题是需要对每个缓存的数据专门维护一个计数器，每次访问都要更新，但这样做会带来高昂的维护开销；另一个问题是不便于处理随时间变化的热度变化，譬如某个曾经频繁访问的数据现在不需要了，但很难自动将它清理出缓存。
- **TinyLFU(Tiny Least Frequently Used)**: TinyLFU是LFU的改进版本。较为复杂。
- **W-TinyLFU(Windows-TinyLFU)**: W-TinyLFU也是TinyLFU的改进版本。很复杂。

I 扩展功能

除了基础功能外，一般还会考虑其他扩展功能，如^[16-6]:

- **加载器**: 许多缓存都有“CacheLoader”之类的设计，加载器可以让缓存从只能被动存储外部放入的数据，变为能够主动通过加载器去加载指定Key值的数据，加载器也是实现自动刷新功能的基础前提。
- **淘汰策略**: 有些缓存的淘汰策略是固定的，也有一些缓存能够支持用户自己根据需要选择不同的淘汰策略。
- **失效策略**: 要求缓存的数据在一定时间后自动失效（移出缓存）或者自动刷新（使用加载器重新加载）。
- **事件通知**: 缓存可能会提供一些事件监听器，让你在数据状态变动（如失效、刷新、移除）时进行一些额外操作。有的缓存还提供了对缓存数据本身的监视能力（Watch功能）。
- **并发级别**: 对于通过分段加锁来实现的缓存（以Guava Cache为代表），往往会提供并发级别的设置，可以简单将其理解为缓存内部是使用多个Map来分段存储数据的，并发级别用于计算出使用Map的数量。如果将并发级别这个参数设置得过大，会引入更多的Map，需要额外维护这些Map而导致更大的时间和空间上的开销；如果设置得过小，又会导致在访问时产生线程阻塞，因为多个线程更新同一个ConcurrentMap的同一个值时会产生锁竞争。
- **容量控制**: 缓存通常都支持指定初始容量和最大容量，初始容量的目的是减少扩容频率，这与Map接口本身的初始容量含义是一致的。最大容量类似于控制Java

堆的-Xmx参数，当缓存接近最大容量时，会自动清理低价值的数据。

- 引用方式：支持将数据设置为软引用或者弱引用，提供引用方式的设置是为了将缓存与Java虚拟机的垃圾收集机制联系起来。
- 统计信息：提供诸如缓存命中率、平均加载时间、自动回收计数等统计信息。
- 持久化：支持将缓存的内容存储到数据库或者磁盘中，进程内缓存提供持久化功能的作用不大，但分布式缓存大多都会考虑提供持久化功能。

下图是主流产品的缓存扩展功能：

表4-4 几款主流进程内缓存方案对比

	ConcurrentHashMap	Ehcache	Guava Cache	Caffeine
访问性能	最高	一般	良好	优秀 接近于 ConcurrentHashMap
淘汰策略	无	支持多种淘汰策略， 如 FIFO、LRU、LFU 等	LRU	W-TinyLFU
扩展功能	只提供基础的访问 接口	并发级别控制； 失效策略； 容量控制； 事件通知； 统计信息；	大致同左	大致同左

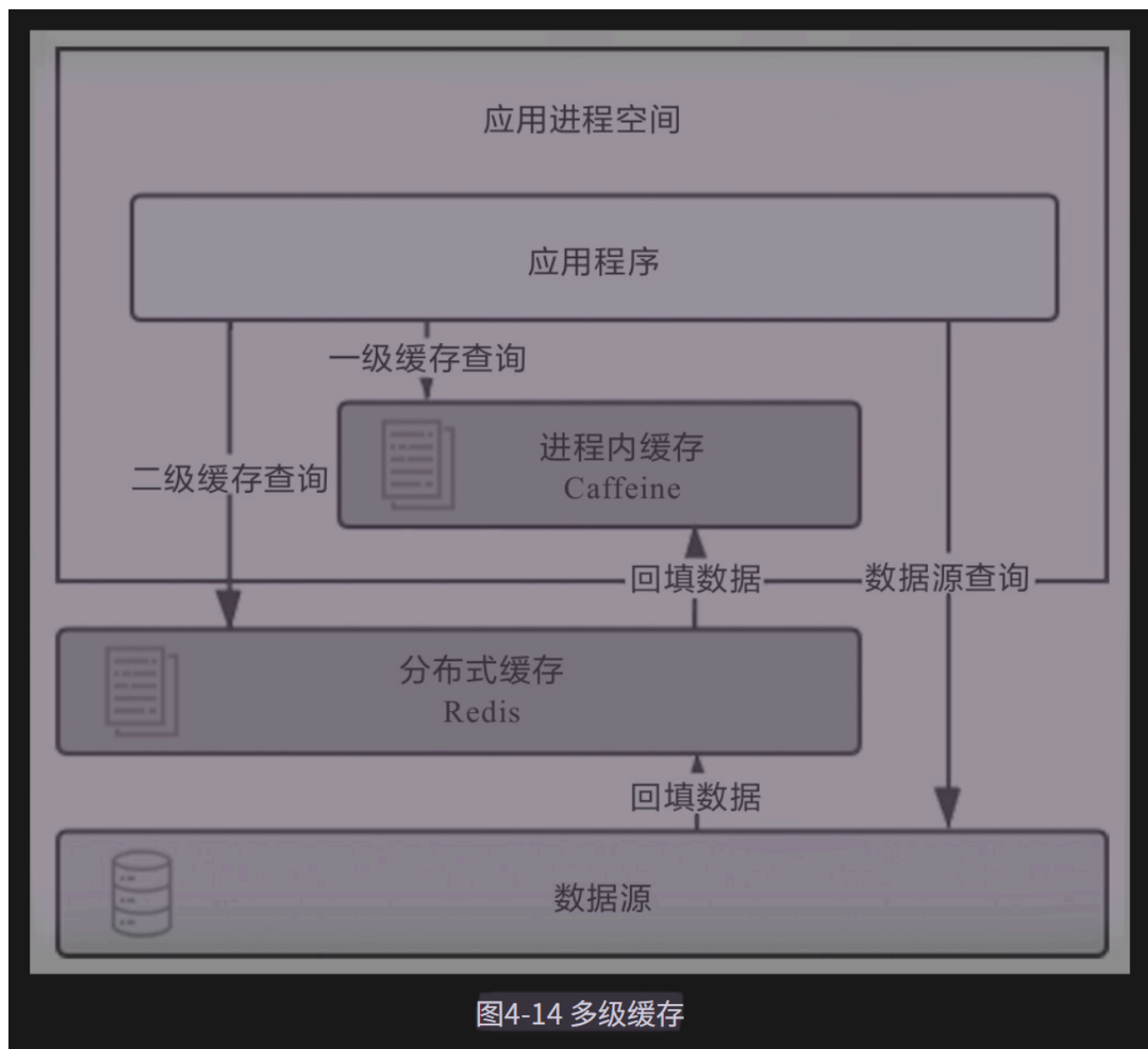
I 分布式缓存

从访问的角度来说，如果是频繁更新但甚少读取的数据，通常是不会有人把它拿去做缓存的，因为这样做没有收益。对于甚少更新但频繁读取的数据，理论上更适合做复制式缓存；对于更新和读取都较为频繁的数据，理论上更适合做集中式缓存^[16-7]。

以下是两种缓存方式的工作原理和代表产品：^[16-8]

- 复制式缓存：复制式缓存可以看作“能够支持分布式的进程内缓存”，它的工作原理与Session复制类似。缓存中所有数据在分布式集群的每个节点里面都有一份副本，读取数据时无须网络访问，直接从当前节点的进程内存中返回，理论上可以做到与进程内缓存一样高的读取性能；但当数据发生变化时，就必须遵循复制协议，将变更同步到集群的每个节点中，复制性能随着节点的增加呈现平方级下降，变更数据的代价十分高昂。复制式缓存的代表是JBossCache、Infinispan。
- 集中式缓存：集中式缓存是目前分布式缓存的主流形式，它的读、写都需要网络访问，好处是不会随着集群节点数量的增加而产生额外的负担，坏处是读、写都不再可能达到进程内缓存那样的高性能。集中式缓存的唯一选择是Redis。

分布式缓存与进程内缓存各有所长，也各有局限，它们是互补而非互斥的关系，如有需要，完全可以将两者搭配使用，构成透明多级缓存(Transparent Multilevel Cache，TMC)。如下图所示。[\[16-9\]](#)



缓存风险

使用缓存会存在风险，主要有[\[17\]](#):

- 缓存穿透
- 缓存击穿
- 缓存雪崩
- 缓存污染

I 缓存穿透

缓存的目的是缓解CPU或者I/O的压力，譬如对数据库做缓存，大部分流量都从缓存中直接返回，只有缓存未能命中的数据请求才会流到数据库中，这样数据库压力自然就减小了。但是如果查询的数据在数据库中根本不存在，缓存里自然也不会有，这类请求的流量每次都不会命中，且每次都会触及末端的数据库，缓存就起不到缓解压力的作用了，这种查询不存在的数据的现象被称为缓存穿透。[\[17-1\]](#)

缓存穿透有可能是业务逻辑本身就存在的固有问题，也有可能是恶意攻击所导致。为了解决缓存穿透问题，通常会采取下面两种办法。[\[17-2\]](#)

- 对于业务逻辑本身不能避免的缓存穿透，可以约定在一定时间内对返回为空的Key值进行缓存（注意是正常返回但是结果为空，不应把抛异常的也当作空值来缓存），使得在一段时间内缓存最多被穿透一次。如果后续业务在数据库中对该Key值插入了新记录，那应当在插入之后主动清理掉缓存的Key值。如果业务时效性允许的话，也可以对缓存设置一个较短的超时时间来自动处理。
- 对于恶意攻击导致的缓存穿透，通常会在缓存之前设置一个**布隆过滤器**来解决。所谓恶意攻击是指请求者刻意构造数据库中肯定不存在的Key值，然后发送大量请求进行查询。布隆过滤器是用最小的代价来判断某个元素是否存在于某个集合的办法。如果布隆过滤器给出的判定结果是请求的数据不存在，直接返回即可，连缓存都不必去查。虽然维护布隆过滤器本身需要一定的成本，但比起攻击造成的资源损耗仍然是值得的。

I 缓存击穿

我们都知道缓存的基本工作原理是首次从真实数据源加载数据，完成加载后回填入缓存，以后其他相同的请求就从缓存中获取数据，以缓解数据源的压力。如果缓存中某些热点数据忽然因某种原因失效了，譬如由于超期而失效，此时又有多个针对该数据的请求同时发送过来，这些请求将全部未能命中缓存，到达真实数据源中，导致其压力剧增，这种现象被称为缓存击穿。要避免缓存击穿问题，通常会采取下面两种办法。[\[17-3\]](#)

- 加锁同步，以请求该数据的Key值为锁，使得只有第一个请求可以流入真实的数据源中，对其他线程则采取阻塞或重试策略。如果是进程内缓存出现问题，施加普通互斥锁即可，如果是分布式缓存中出现问题，就施加分布式锁，这样数据源就不会同时收到大量针对同一个数据的请求了。

- 热点数据由代码来手动管理。缓存击穿是仅针对热点数据自动失效才引发的问题，对于这类数据，可以直接由开发者通过代码来有计划地完成更新、失效，避免由缓存的策略

| 缓存雪崩

缓存击穿是针对单个热点数据失效，由大量请求击穿缓存而给真实数据源带来压力。还有一种可能更普遍的情况，即不是针对单个热点数据的大量请求，而是由于大批不同的数据在短时间内一起失效，导致这些数据的请求都击穿缓存到达数据源，同样令数据源在短时间内压力剧增。[\[17-4\]](#)

出现这种情况，往往是因为系统有专门的缓存预热功能，或者大量公共数据是由某一次冷操作加载的，使得由此载入缓存的大批数据具有相同的过期时间，在同一时刻一起失效；也可能是因为缓存服务由于某些原因崩溃后重启，造成大量数据同时失效。这种现象被称为缓存雪崩。要避免缓存雪崩问题，通常会采取下面三种办法。[\[17-5\]](#)

- 提升缓存系统可用性，建设分布式缓存的集群。
- 启用透明多级缓存，这样各个服务节点一级缓存中的数据通常会具有不一样的加载时间，也就分散了它们的过期时间。
- 将缓存的生存期从固定时间改为一个时间段内的随机时间，譬如原本是1h过期，在缓存不同数据时，可以设置生存期为55min到65min之间的某个随机时间。

| 缓存污染

缓存污染是指缓存中的数据与真实数据源中的数据不一致的现象。[\[17-6\]](#)

缓存污染多数是由开发者更新缓存不规范造成的，譬如你从缓存中获得了某个对象，更新了对应的属性，但最后因为某些原因，譬如后续业务发生异常回滚了，最终没有成功写入数据库，导致缓存的数据是新的，数据库中的数据是旧的。为了尽可能地提高使用缓存时的一致性，目前已经有很多更新缓存时可以遵循的设计模式，譬如 Cache Aside、Read/Write Through、Write Behind Caching等。其中最简单、成本最低的Cache Aside模式是指：[\[17-7\]](#)

- 读数据时，先读缓存，如果没有，再读数据源，然后将数据放入缓存，再响应请求；
 - 写数据时，先写数据源，然后失效（而不是更新）掉缓存。
-

1. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.1 客户端缓存 ↩
2. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.1.1 强制缓存 ↩↩↩↩
3. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.1.2 协商缓存 ↩↩↩↩↩↩↩↩↩↩
4. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.2 域名解析 ↩
5. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.3.1 连接数优化 ↩↩↩↩
6. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.3.2 传输压缩 ↩↩
7. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.4.1 路由解析 ↩
8. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.4.2 内容分发 ↩↩↩
9. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.4.3 CDN应用 ↩
10. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.5 负载均衡 ↩
11. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.5.1 数据链路层负载均衡 ↩
12. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.5.2 网络层负载均衡 ↩↩↩↩↩↩↩
13. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.5.3 应用层负载均衡 ↩↩
14. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.5.4 均衡策略与实现 ↩↩↩
15. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.6 服务器缓存 ↩
16. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.6.1 缓存属性 ↩↩↩↩↩↩↩↩↩↩
17. 《凤凰架构-构建可靠的大型分布式系统》，周志明，4.6.2 缓存风险 ↩↩↩↩↩↩↩↩